Reporte de Práctica: Implementación de una ALU de 5 bits en FPGA

Mauricio Morán Riestra
Patricio Funes San Martín
Universidad de Las Americas Puebla (UDLAP)
Laboratorio de procesamiento de señales

27 de septiembre de 2026

Resumen

En esta práctica se diseña e implementa una Unidad Aritmético–Lógica (ALU) de 5 bits utilizando una descripción estructural basada en medios sumadores (HA), sumadores completos (FA), sumadores en cascada (*Ripple Carry*), medios restadores (HS), restadores completos (FS), restadores en cascada y multiplicación binaria. El resultado de la operación es convertido de binario a BCD y desplegado en formato decimal mediante los displays de 7 segmentos de la tarjeta Terasic DE10-Lite. La validación del diseño se realiza primero en simulación (Active-HDL) y posteriormente se compila e implementa en la $FPGA$ usando Quartus Lite.

1. Objetivo

- Diseñar e implementar una Unidad Aritmético–Lógica (ALU) de 5 bits usando lenguaje Verilog y la plataforma FPGA DE10-Lite.
- Realizar operaciones de suma, resta y multiplicación entre dos operandos con signo
- Mostrar el resultado en formato decimal mediante displays de 7 segmentos.
- Simular el funcionamiento del sistema en **Active-HDL** (pruebas funcionales) y luego compilar/implementar en **Quartus Lite** para la DE10-Lite.

2. Estado del Arte

Las Unidades Aritmético-Lógicas son el núcleo de procesamiento en procesadores y microcontroladores modernos. En sistemas embebidos, el desarrollo de bloques aritméticos modularizados permite comprender la propagación del acarreo y los costos en recursos lógicos.

El uso de estructuras en cascada combinando Medios Sumadores (HA) y Sumadores Completos (FA) optimiza el uso de compuertas en la primera etapa al no requerir un acarreo de entrada previo ($C_{in} = 0$), mientras que los multiplicadores inferidos por operadores nativos permiten aprovechar bloques de procesamiento dedicados dentro de la arquitectura FPGA.

2.1. Sistemas de aritmética en FPGA

A pesar de que existen arquitecturas avanzadas para acelerar la propagación del acarreo (tales como *Carry-Lookahead* o *Carry-Select*), diversos trabajos en HDL emplean la arquitectura *Ripple-Carry* por su simplicidad en la descripción de la estructura jerárquica.

2.2. Sistemas de Suma

2.2.1. Medio Sumador (*Half Aadder*)

Un medio sumador suma dos bits a y b, produciendo un acarreo de salida C:

$$S = a \oplus b \tag{1}$$

$$C = a \cdot b \tag{2}$$

2.2.2. Sumador completo (*Full Aadder*)

Un sumador completo suma a, b y un acarreo proveniente de una etapa previa C_{in}

$$S = (a \oplus b) \oplus C_{in} \tag{3}$$

$$C_{out} = (a \cdot b) + ((a \oplus b) \cdot C_{in}) \tag{4}$$

2.2.3. Sumador Ripple-Carry de 5 bits

Un sumador de 5 bits puede construirse conectando etapas en cascada mediante la propagación del acarreo, con con un medio sumador (HA) en el bit menos significativo y cuatro sumadores completos (FA) en cascada. A partir del segundo bit, cada etapa requiere una entrada de acarreo C_{in}.

2.3. Sistemas de Resta

2.3.1. Medio Restador (*Half Substractor*)

Un medio sumador resta dos bits a y b, produciendo un préstamo de salida B:

$$D = a \oplus b \tag{5}$$

$$B = \bar{a} \cdot b \tag{6}$$

2.3.2. Restador Completo (*Full Substractor*)

Un restador completo calcula la diferencia y considerando el préstamo B_{in} de la etapa anterior, produciendo un nuevo préstamo B_{out}:

$$D = a \oplus b \oplus B_{in} \tag{7}$$

$$B_{out} = (\bar{a} \cdot b) + ((\bar{a} \oplus b) \cdot B_{in}) \tag{8}$$

2.3.3. Restador Ripple-Carry de 5 bits

Al igual que el sumador, el restador de 5 bits se construye conectando con un medio restador (HS) en el bit menos significativo y cuatro restadores completos (FS) para los bits restantes, propagando el préstamo B_{out} de hacia la siguiente etapa inmediata.

2.4. Multiplicación Binaria

La multiplicación binaria para operandos con signo puede abstraerse mediante sintaxis de alto nivel en HDL. El operador `*` permite que la herramienta de síntesis infiera bloques aritméticos dedicados (*DSP Blocks*) dentro de la matriz del FPGA, siguiendo la lógica:

$$Z = a \cdot b \tag{9}$$

Esta alternativa optimiza los tiempos de propagación y consumo de compuertas frente a arreglos secuenciales.

2.5. Conversión a BCD

El display para el usuario requiere convertir la salida binaria a formato decimal codificado en binario (*BCD*) mediante algoritmos como *Double Dabble* (Shift-Add-3). Posteriormente, cada dígito BCD (unidades, decenas, centenas) se traduce a 7 segmentos (a, b, c, d, e, f, g). Cabe destacar que los displays integrados en la tarjeta DE10-Lite requieren una lógica de activación en bajo (un segmento se enciende cuando el pin del FPGA entrega un '0' lógico y se apaga con un '1').

3. Planteamiento del Experimento

Para la ejecución de la práctica y las pruebas físicas del sistema se empleó el siguiente material:

- **Tarjeta de desarrollo FPGA:** Terasic DE10-Lite (basada en el dispositivo Intel MAX 10 10M50DAF484C7G).
- **Breadboard (Protoboard):** Tabla de pruebas de laboratorio para interconexión externa.
- **Cables de interconexión (*Jumpers*):** Cables tipo Macho-Hembra y Macho-Macho para la asignación de señales físicas desde los conectores de expansión GPIO de 3.3 V hacia la protoboard.
- **Software de diseño y desarrollo:** Quartus Prime Lite Edition (síntesis y asignación de pines) y Active-HDL (simulación previa).

Para determinar lo lógica funcional del código, se decidió representarla a través de un diagrama a bloques, centrado en la distribución de operandos de entrada/salida.

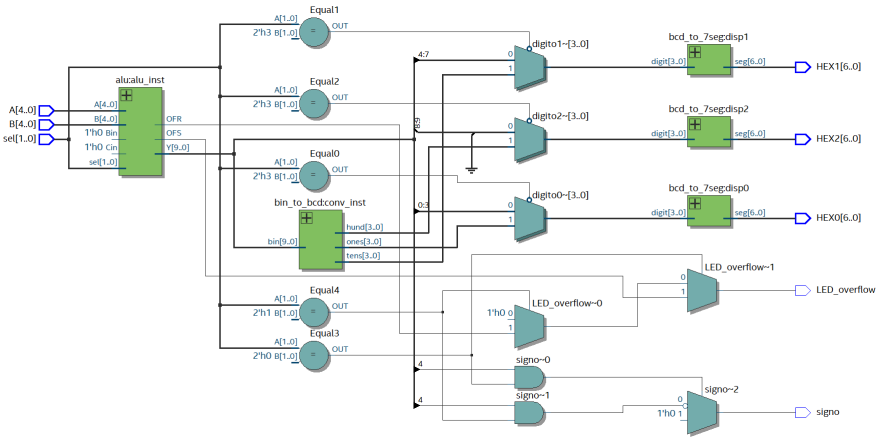


Figura 1: Esquemático de la lógica del programa.

El montaje teórico mostrado en la Figura 1 describe el funcionamiento del sumador y el decodificador. En términos más específicos, el diseño propuesto sigue la siguiente arquitectura:

- **Módulo ALU:** Recibe los vectores de entrada de 5 bits $A[4.,0]$ y $B[4.,0]$, la señal de selección de operación $sel[1.,0]$, y mantiene en cero lógico las entradas iniciales de acarreo y préstamo ($1'b0$ Cin y $1'b0$ Bin). Entrega el bus de resultado general de 10 bits $Y[9.,0]$ (`result`) y las señales de bandera OFS (`overflow1`) y OFR (`overflow2`).
- **Convertidor Binario a BCD:** Toma la salida `result` de la ALU y genera tres buses BCD de 4 bits correspondientes a las centenas (`centenas`), decenas (`decenas`) y unidades (`unidades`).
- **Red de Selección y Multiplexado:** Mediante operadores condicionales ternarios, el módulo superior determina si los buses de entrada hacia los decodificadores (`digito0`, `digito1`, `digito2`) se alimentan del módulo BCD o de los nibbles directos de `result`.
- **Decodificadores a 7 Segmentos:** Las salidas multiplexadas alimentan tres instancias decodificadoras (`disp0`, `disp1`, `disp2`) encargadas de manejar los buses físicos $HEX0[6.,0]$, $HEX1[6.,0]$ y $HEX2[6.,0]$ operando en lógica activa en bajo.
- **Marcadores:** Las salidas indicadoras conmutan la señal `LED_overflow` seleccionando `overflow1` para suma (`2'b00`) y `overflow2` para resta (`2'b01`). Asimismo, el indicador de `signo` evalúa la presencia del bit $result[4]$ activo durante la suma y la resta.

Para operandos de con signo negativo, el rango de valores está delimitado por:

$$-2^{n-1} \leq N \leq 2^{n-1} - 1 \implies -16 \leq N \leq 15 \tag{10}$$

Donde el bit $A[4]$ representa el signo (MSB).

En cunato a otras especificaciones:

- Los 5 bits de cada operando (A y B) se asignaron respectivamente a los 10 switches físicos integrados de la FPGA, operando con lógica *activo en alto*.

- Los 2 bits del Selector se direccionaron a los pines del cabezal de expansión GPIO de 2x20.
- El marcador para los casos de `overflow` se implemento utilizando uno de los leds integrados en el FPGA.
- Mediante cables *jumper*, dichos pines se conectaron a la línea de tierra (`GND`) de la protoboard para forzar las entradas a "0" lógico de forma predeterminada. La desconexión física del *jumper* permitió simular un estado "1" lógico mediante resistencias de *pull-up* internas del FPGA.

4. Casos de Estudio o Pruebas

Para poner a prueba la FPGA, se definieron tres escenarios críticos de evaluación. El proceso de validación se dividió en dos etapas consecutivas: el modelado virtual adaptativo y la posterior experimentación en hardware.

4.1. Validación por Simulación

Antes de las pruebas físicas, el comportamiento del sistema se validó de forma virtual. Los tres vectores de prueba representativos se introdujeron mediante un bloque de estímulos (*testbench*) dentro del entorno de desarrollo **Active-HDL**.

Las formas de onda obtenidas en el analizador lógico digital confirmaron el correcto funcionamiento del diseño. A partir de los resultados, se realizó la siguiente tabla.

a	b	Operación (`sel`)	Resultado Teórico	`LED_overflow`	`signo`
-16	-1	Suma (`00`)	-17 (Overflow)	1	0
-16	-1	Resta (`01`)	-15	0	1
-16	-1	Multiplicación (`10`)	16	0	0
-7	-10	Suma (`00`)	-17 (Overflow)	1	0
-7	-10	Resta (`01`)	3	0	0
-7	-10	Multiplicación (`10`)	70	0	0
7	9	Suma (`00`)	16 (Overflow)	1	0
7	9	Resta (`01`)	-2	0	1
7	9	Multiplicación (`10`)	63	0	0

Tabla 1: Resultados teóricos y comportamiento de banderas para los casos de estudio.

La simulación demostró el comportamiento del sistema ante números negativos y positivos en complemento a dos dentro del FPGA, evaluando tres parejas de operandos (a y b) ejecutando las tres operaciones disponibles (suma, resta y multiplicación).

4.2. Pruebas en Hardware

Una vez comprobada la estabilidad del diseño en el simulador, se procedió a la carga del archivo de programación en la tarjeta DE10-Lite mediante **Quartus Lite**, aplicando los siguientes casos de estudio de manera controlada:

- **Prueba 1 (-16 y -1)**

- **Entrada A:** 11111 (-1)
- **Entrada B:** 10000 (-16)
- **Suma (`sel`: 00):**

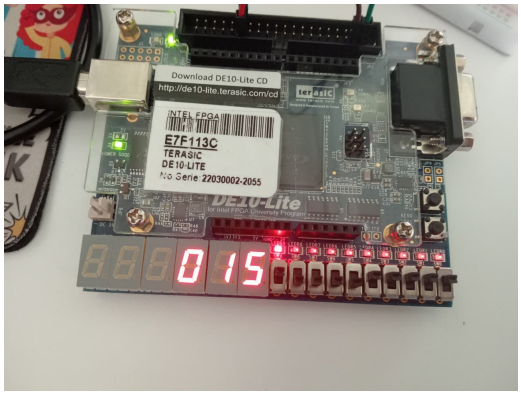


Figura 2: Suma entre los operandos -16 y -1, con el marcador de `overflow` activo.

- **Resta (`sel`: 01)**:

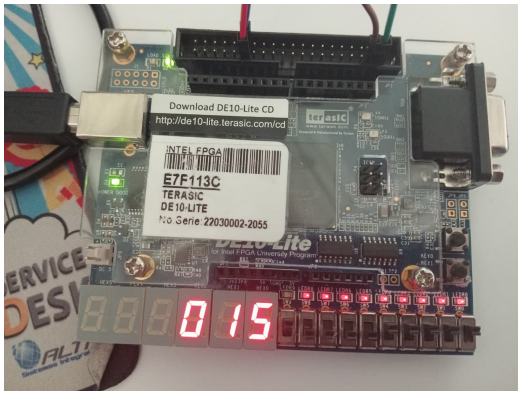


Figura 3: Resta entre los operandos -16 y -1.

- **Multiplicación (`sel`: 10)**:

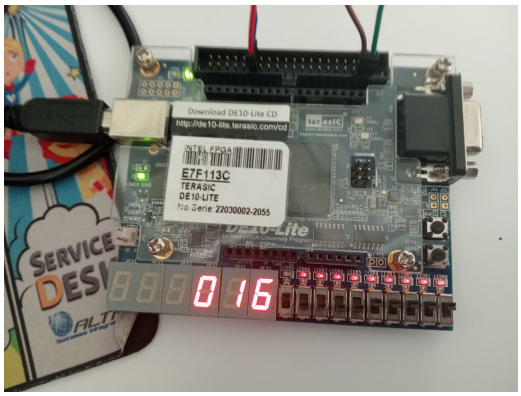


Figura 4: Multiplicación entre los operandos -16 y -1.

- **Prueba 2 (-7 y -10)**

- **Entrada A:** 11001 (-7)
- **Entrada B:** 10110 (-10)

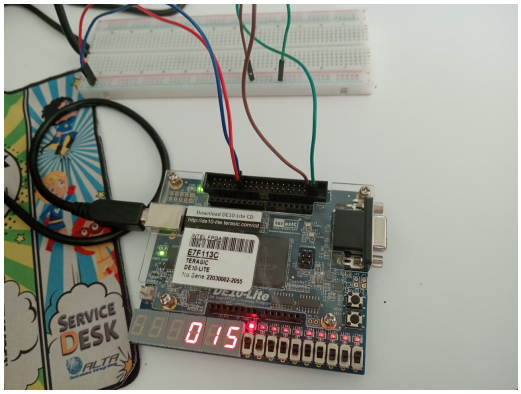


Figura 5: Suma entre los operandos -7 y -10, con el marcador de `overflow` activo.

- **Resta (`sel`: 01)**:

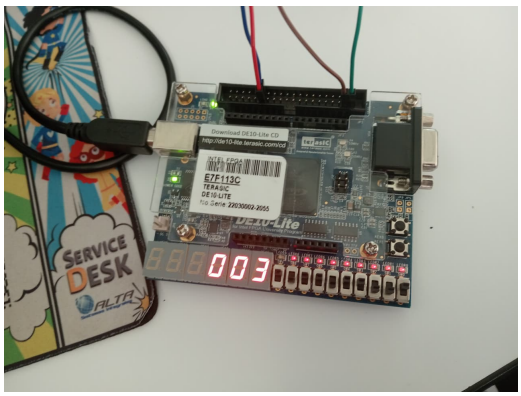


Figura 6: Resta entre los operandos -7 y -10.

- **Multiplicación (`sel`: 10)**:

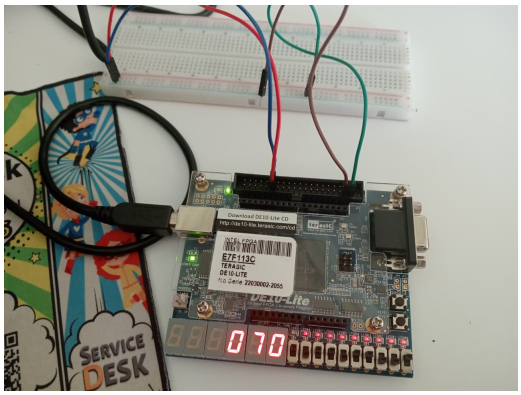


Figura 7: Multiplicación entre los operandos -7 y -10.

- **Prueba 3 (7 y 9)**
  - **Entrada A:** 00111 (7)

- **Entrada B:** 01001 (9)

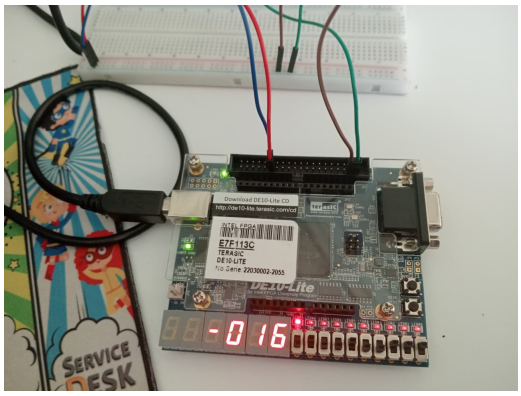


Figura 8: Suma entre los operandos 7 y 9, con el marcador de `overflow` activo.

- **Resta (`sel`: 01)**:

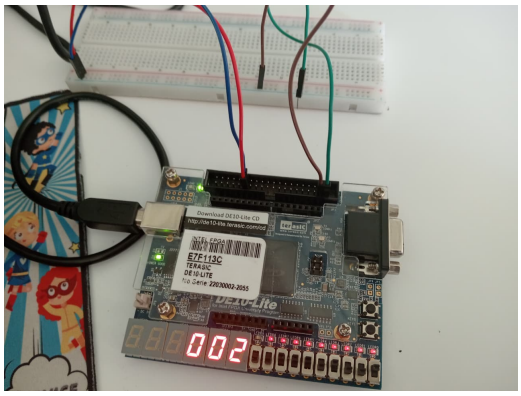


Figura 9: Resta entre los operandos 7 y 9.

- **Multiplicación (`sel`: 10)**:

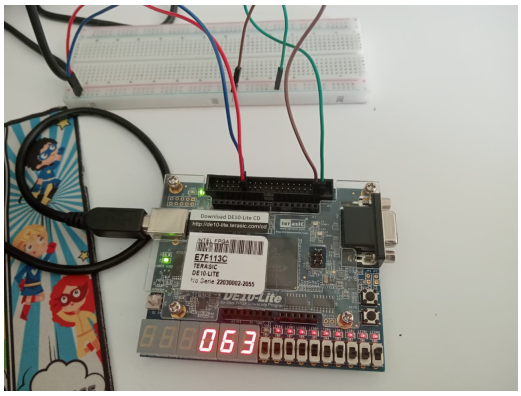


Figura 10: Multiplicación entre los operandos 7 y 9.

5. Análisis de Resultados

La validación de la ALU de 5 bits se realizó mediante tres casos de prueba, evaluando las operaciones de suma, resta y multiplicación para operandos positivos y negativos

representados en complemento a dos. Los resultados obtenidos físicamente en la tarjeta DE10-Lite coincidieron con los valores esperados establecidos previamente durante la simulación:

- **Prueba 1 (-16 y -1):** e utilizaron los operandos -16 y -1. Para la operación de suma se obtuvo un resultado aritmético de -17, valor que se encuentra fuera del rango representable con signo mediante 5 bits, el cual está comprendido entre -16 y 15. Por esta razón, el indicador de overflow se activó durante esta operación. En la resta, el resultado esperado fue -15, mientras que en la multiplicación se obtuvo 16. Las pruebas realizadas en hardware mostraron el comportamiento correspondiente para las tres operaciones seleccionadas.
- **Prueba 2 (-7 y -10):** Al La suma produjo un resultado aritmético de -17, generando nuevamente una condición de overflow debido a que este valor se encuentra fuera del intervalo permitido para una representación con signo de 5 bits. En la operación de resta se obtuvo 3, mientras que la multiplicación produjo 70. En las pruebas físicas de la FPGA, los displays mostraron los valores correspondientes a estas operaciones, incluyendo 003 para la resta y 070 para la multiplicación.
- **Prueba 3 (9 y 7):** La suma produjo 16, lo que provocó una condición de overflow debido a que el máximo valor positivo representable con 5 bits con signo es 15. La resta generó un resultado de 2, mientras que la multiplicación produjo 63. Durante las pruebas físicas se observaron los valores 016, 002 y 063 en los displays para las operaciones correspondientes.

6. Conclusiones

En esta práctica se implementó una Unidad Aritmético-Lógica de 5 bits mediante una estructura jerárquica en Verilog, integrando bloques básicos de suma y resta, así como una operación de multiplicación. El diseño permitió trabajar con operandos positivos y negativos representados en complemento a dos, además de incorporar indicadores para condiciones de overflow y signo. De igual manera, se integró un sistema de conversión y visualización mediante los displays de 7 segmentos de la tarjeta DE10-Lite.

La validación mediante simulación en Active-HDL y las pruebas posteriores en hardware permitieron comprobar el comportamiento de las tres operaciones implementadas bajo diferentes combinaciones de operandos. Los casos seleccionados incluyeron resultados dentro y fuera del rango representable con 5 bits con signo, permitiendo observar la activación del indicador de overflow en operaciones como $-16 + (-1)$, $-7 + (-10)$ y $7 + 9$. Asimismo, las operaciones de resta y multiplicación produjeron los resultados esperados durante las pruebas físicas realizadas en la FPGA

Referencias

[1] M. M. Mano & M. D. Ciletti, *Digital Design (6ta ed.)*. Pearson, 2017.

[2] J. F. Wakerly, *Digital Design: Principles and Practices (4ta ed.)*. Pearson/Prentice Hall, 2006.

[3] S. Brown & Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design (3ra ed.)*. McGraw-Hill Education, 2014.

[4] Terasic Inc., *DE10-Lite User Manual*, 2016.